

Week 8

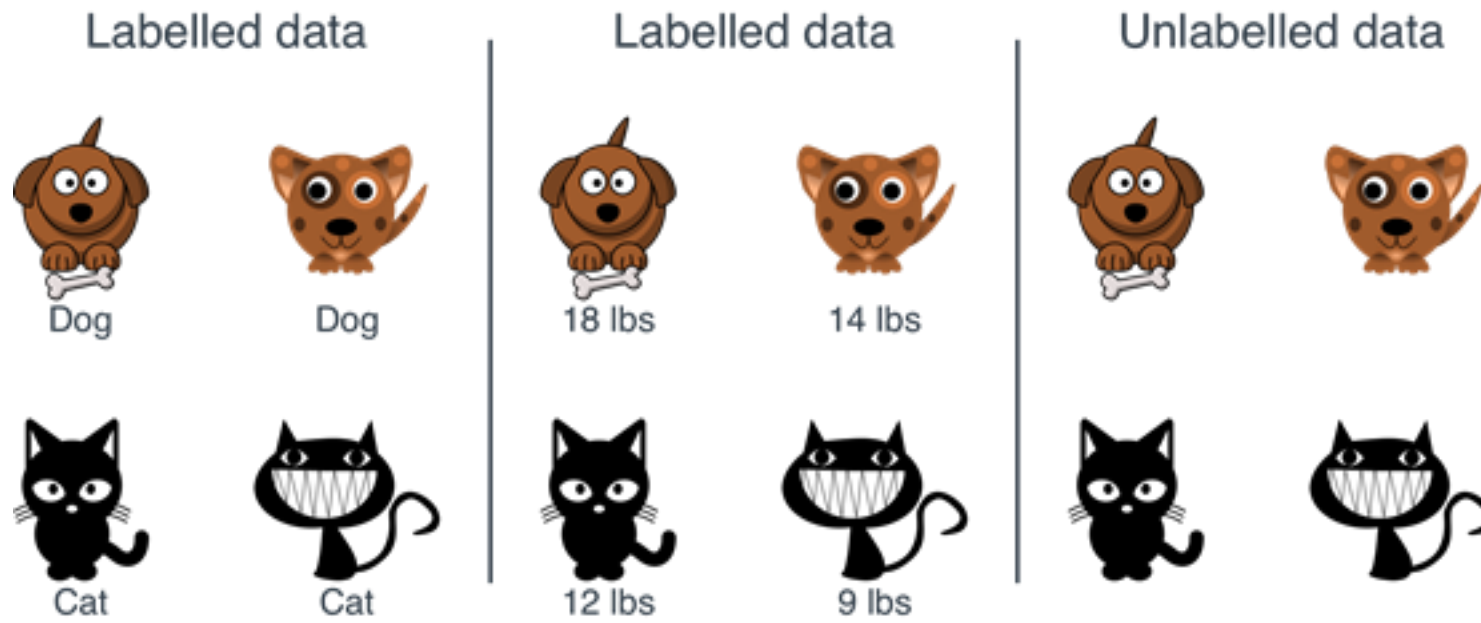
Agenda

- K-means
- KNN
- PCA

Unsupervised Learning

Although most of the applications of Machine Learning today are based on supervised learning (and as a result, this is where most of the investments go to), the vast majority of the available data is unlabeled: we have the input features $\mathbf{X}$, but we do not have the labels $\mathbf{y}$.

3 / 50



The Goals of Unsupervised Learning

The goal is to discover interesting things about the data:

Visualization:

Is there an informative way to visualize the data?

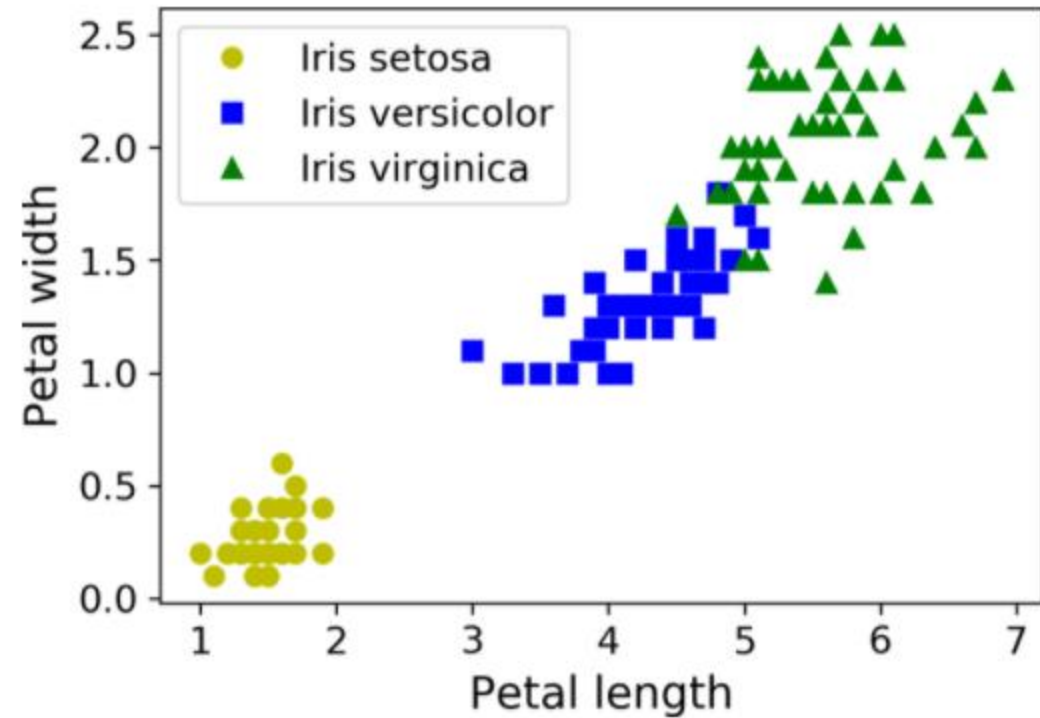
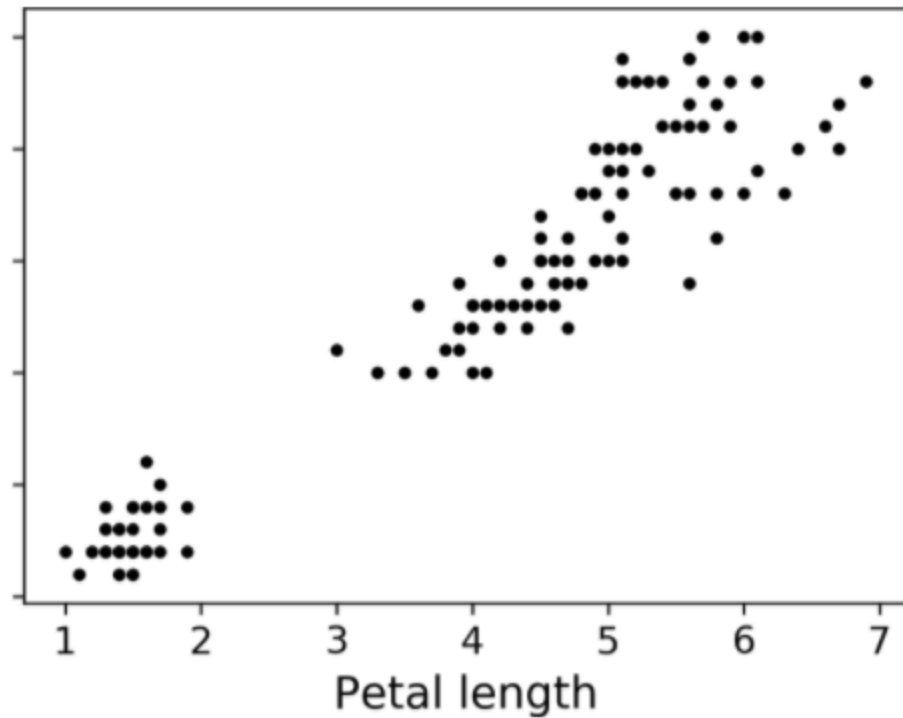
Data Insights

Can we discover subgroups among the variables or among the observations?

Are the subgroups meaningful?

Supervised Learning example

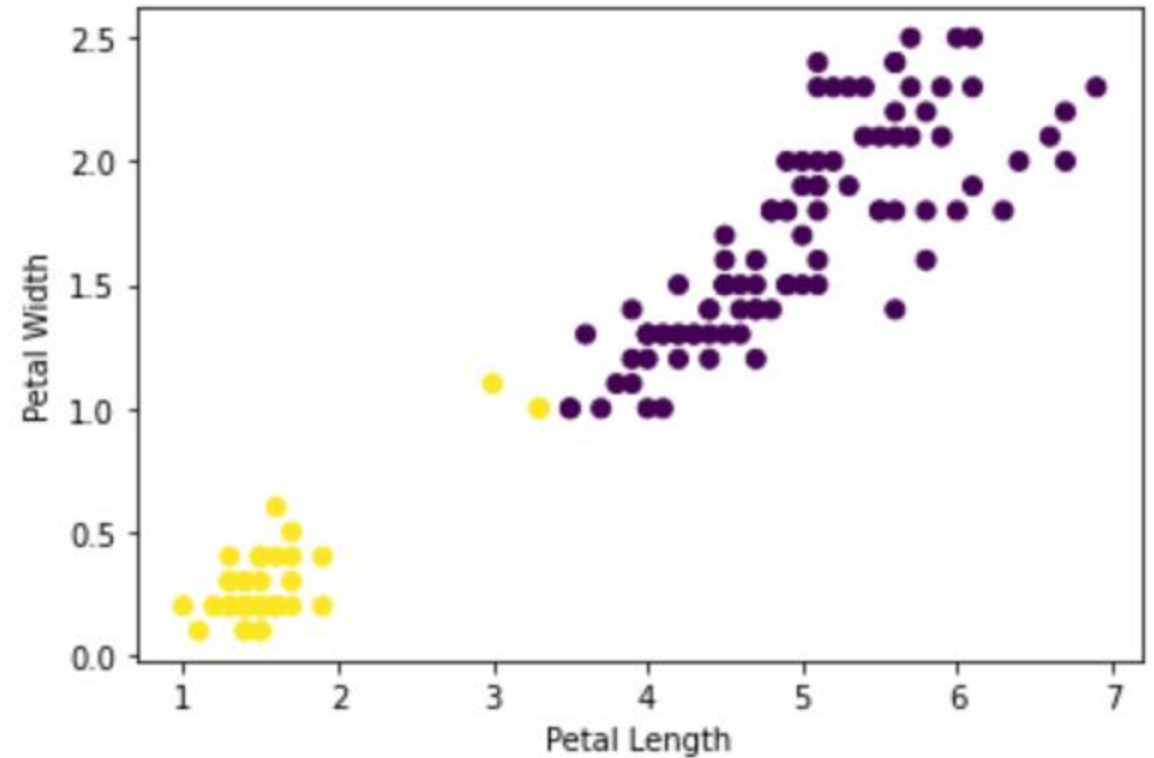
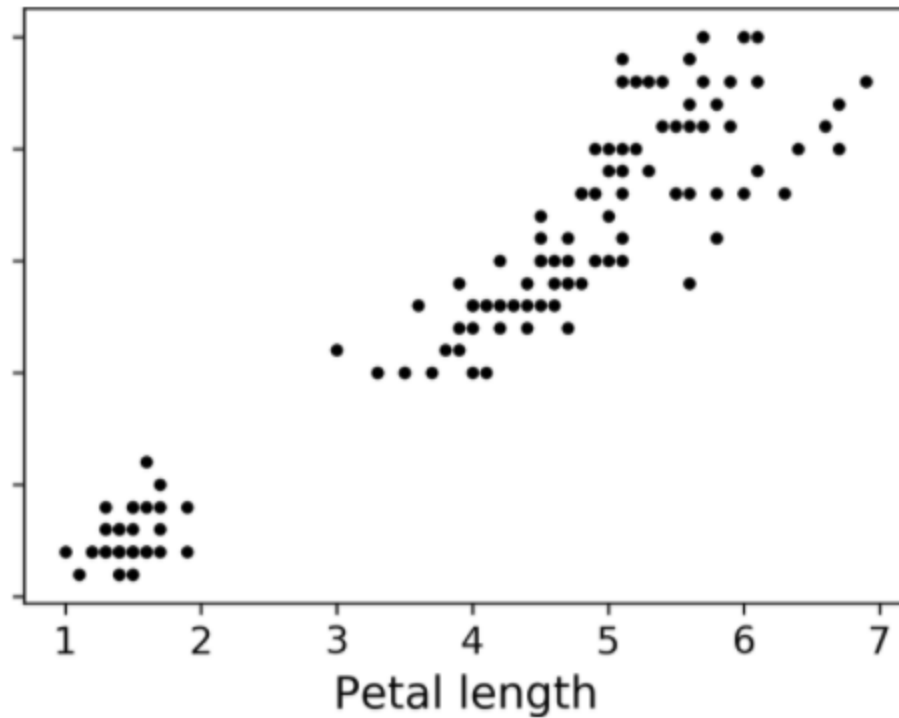
The flower labels are known and can be visualized



Unsupervised Learning example

Given the iris dataset we can group similar points and assign it to a cluster.

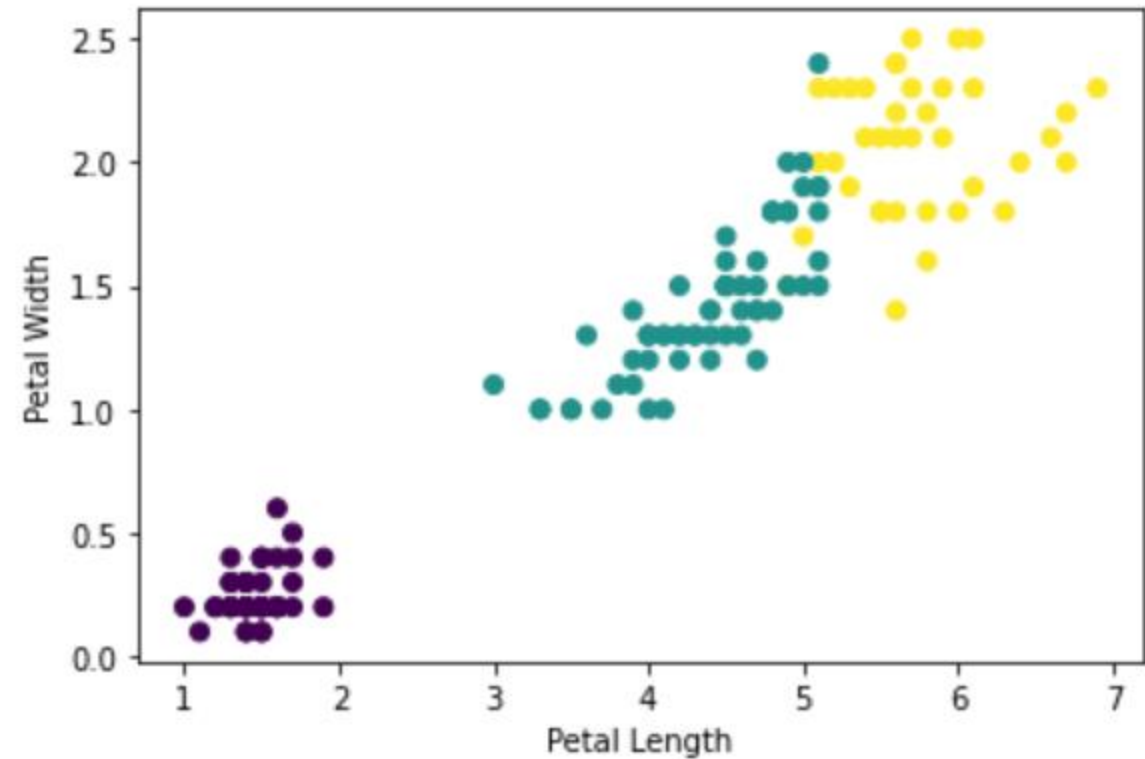
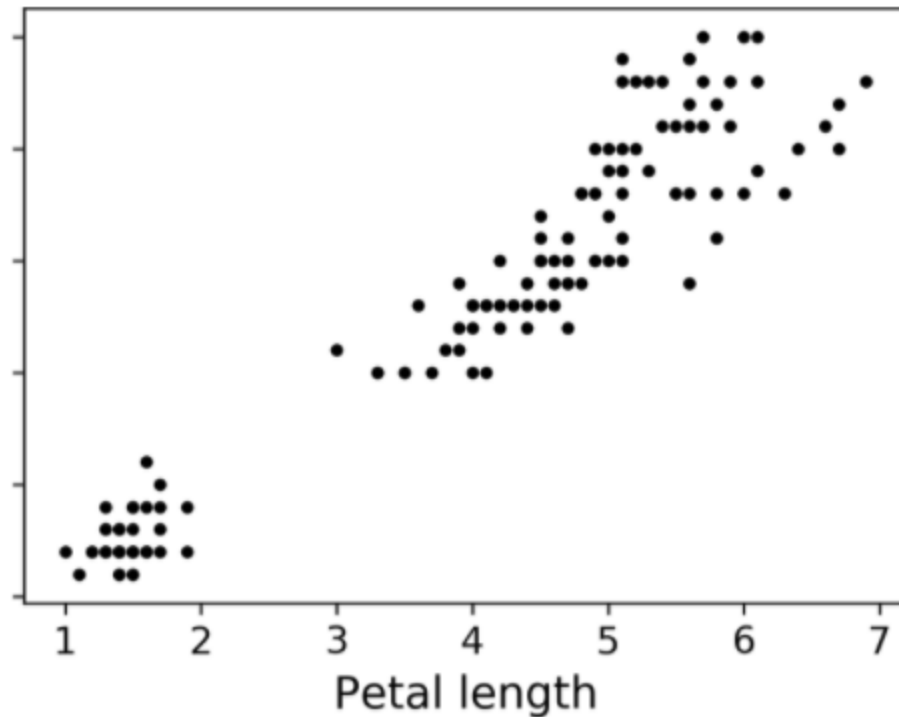
2 Clusters



Unsupervised Learning example

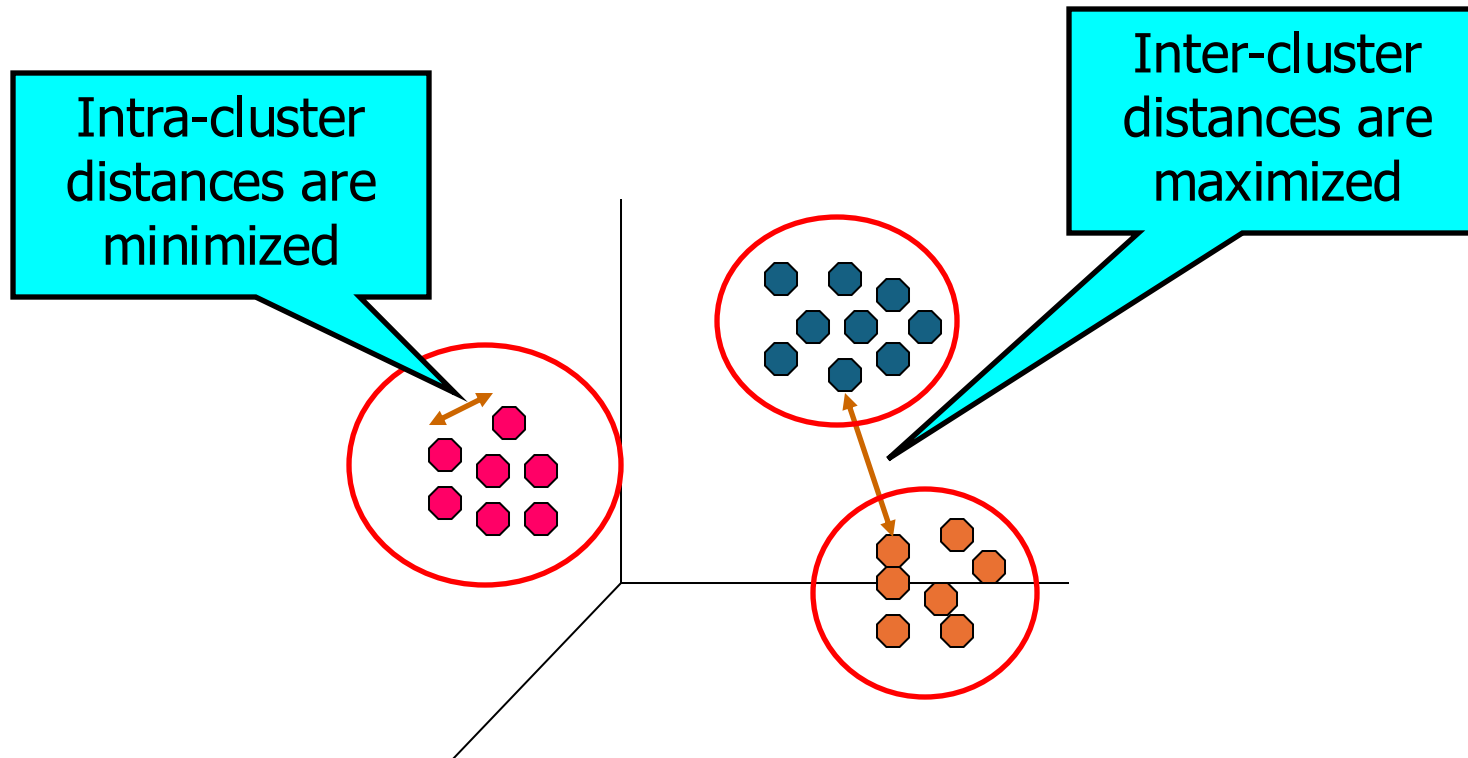
Given the iris dataset we can group similar points and assign it to a cluster.

3 Clusters



Unsupervised algorithm: Clustering

- Goal: find groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in other groups



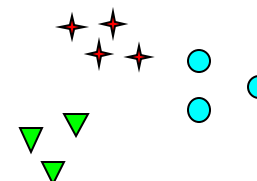
Applications of Cluster Analysis

- **Customer Segmentation** – used in recommender systems – recommend products that other users in same cluster bought.
- **Data analysis** – run analysis on each cluster separately
- **Anomaly detection** – a sample that is far from any cluster is likely an anomaly. Used in fraud detection, manufacturing defects, network security.
- **Semi-supervised learning** – used when missing labels. Propagate labels to missing samples in same cluster.
- **Search engines** – used when searching images. Images are part of a cluster. Show similar images.

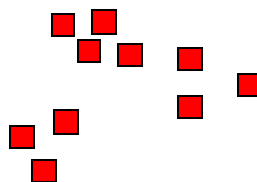
Notion of a Cluster can be Ambiguous



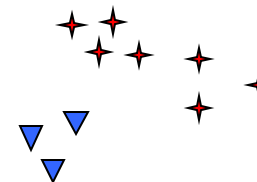
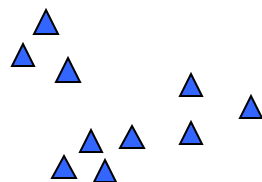
How many clusters?



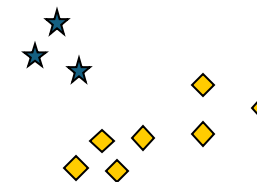
Six Clusters



Two Clusters



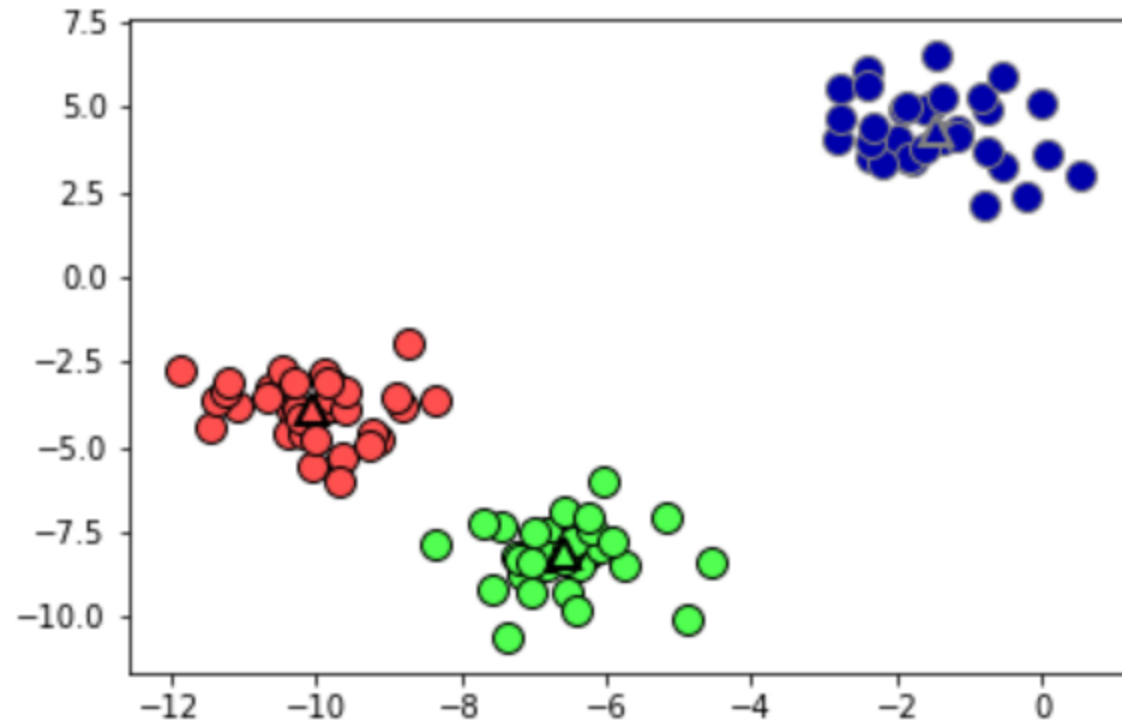
Four Clusters



Types of Clusters: Well-Separated

- Well-Separated Clusters:

A cluster is a set of points such that any point in a cluster is closer (or more similar) to every other point in the cluster than to any point not in the cluster.



3 well-separated clusters

Types of Clusters: Unbalanced

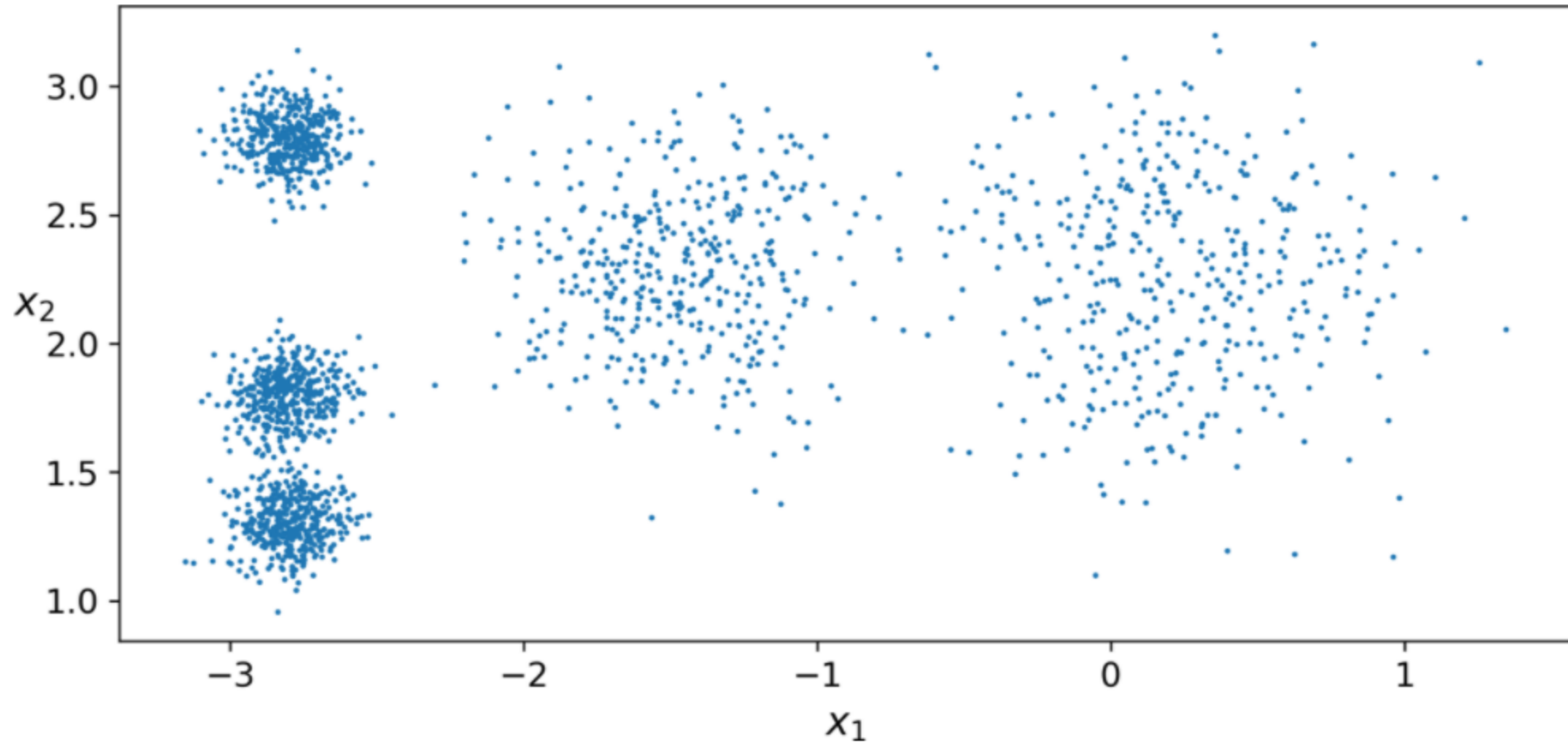


Figure 9-2. An unlabeled dataset composed of five blobs of instances

Clustering Algorithms

- **K-Means** - Partitions data into k clusters by minimizing variance within each cluster. Fast and widely used, but requires specifying the number of clusters beforehand.
- **Hierarchical Clustering** - Builds a tree of clusters either bottom-up (agglomerative) or top-down (divisive). Produces a dendrogram showing relationships at different scales.
- **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) - Groups together points that are closely packed and marks outliers as noise. Doesn't require specifying the number of clusters.
- **Gaussian Mixture Models (GMM)** - Assumes data comes from a mixture of Gaussian distributions. Uses probabilistic assignment rather than hard clustering.
- **Mean Shift** - Finds clusters by locating density maxima. Automatically determines the number of clusters based on the data.
- **Spectral Clustering** - Uses eigenvalues of a similarity matrix to reduce dimensions before clustering. Effective for complex, non-convex cluster shapes.
- **OPTICS (Ordering Points To Identify Clustering Structure)** - Similar to DBSCAN but better handles varying density clusters by creating an augmented ordering of the data.
- **Affinity Propagation** - Identifies exemplars among data points and forms clusters around them through message passing between points.
- **BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies)** - Designed for large datasets, builds a tree structure incrementally to minimize memory usage.

K-means Clustering

Simple iterative algorithm

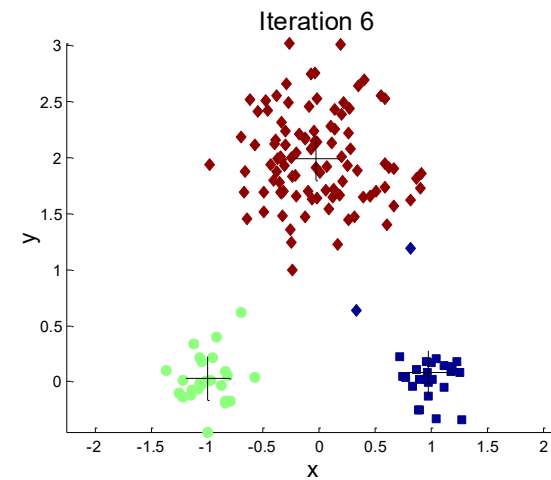
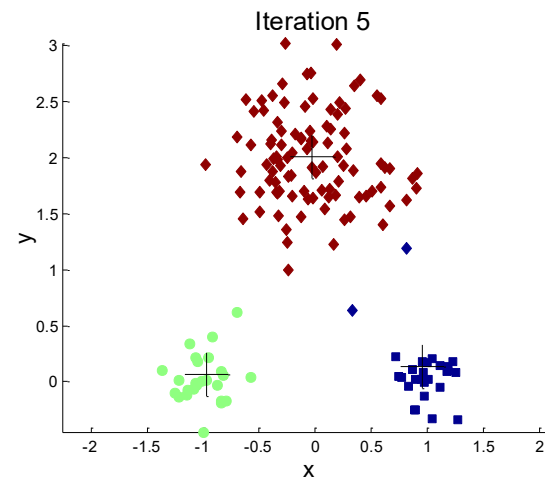
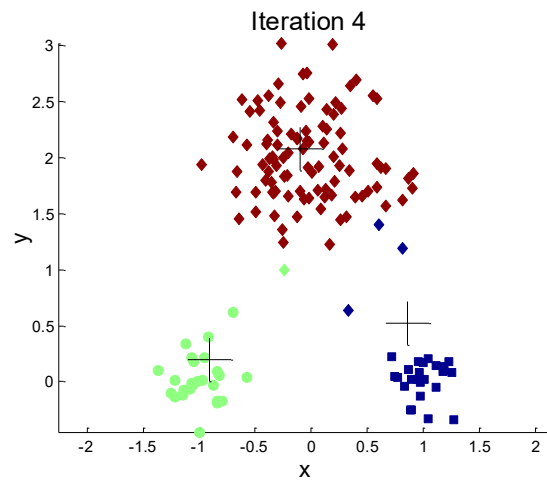
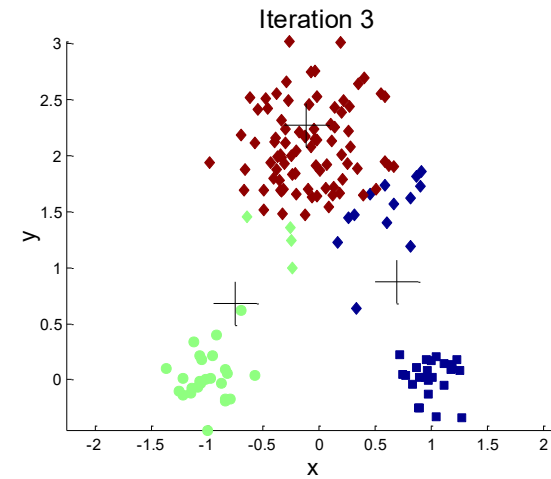
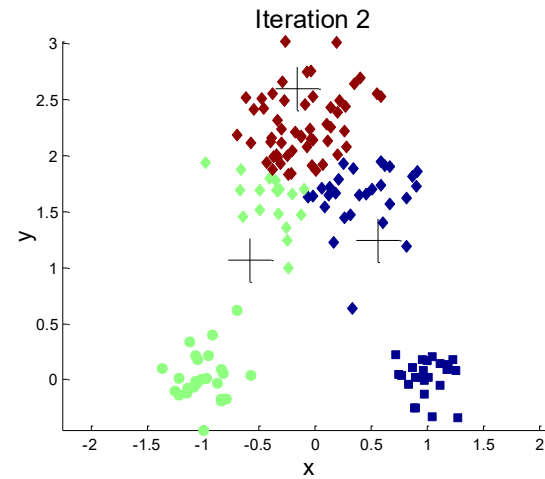
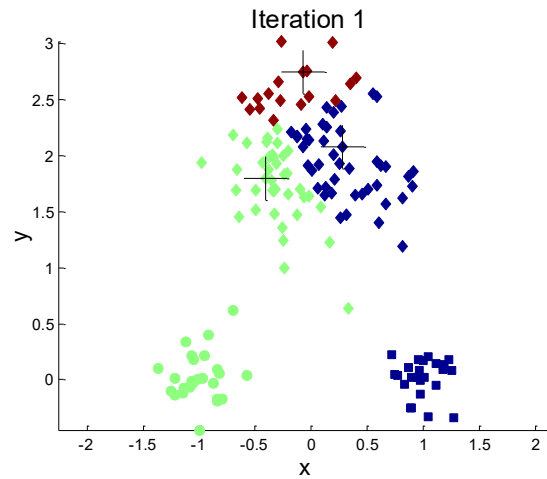
Number of clusters, K , must be specified

- Start algorithm:
- Select k centroids (center point) from the data. These can be chosen randomly.
- Each sample in dataset is assigned to the closest centroid forming k clusters
- Repeat:
 - Compute the average of all points (features) in each cluster and make this the new centroid
 - Reassign points to the cluster with closest centroid
 - Repeat until points are no longer assigned a cluster

K-means Clustering

-
- 1: Select K points as the initial centroids.
 - 2: **repeat**
 - 3: Form K clusters by assigning all points to the closest centroid.
 - 4: Recompute the centroid of each cluster.
 - 5: **until** The centroids don't change
-

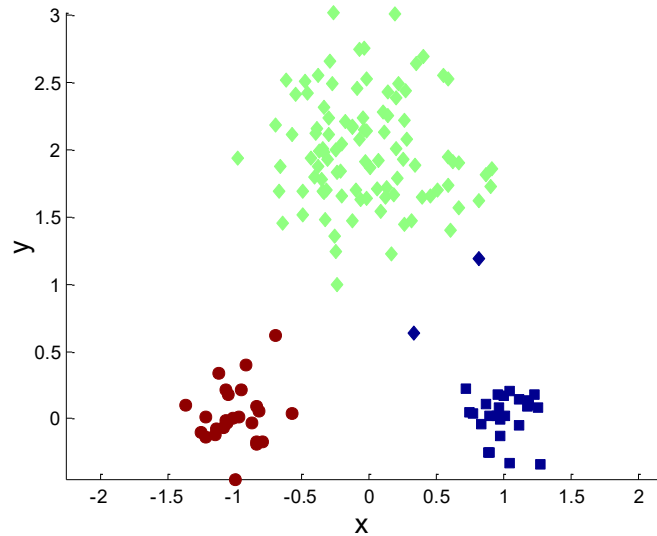
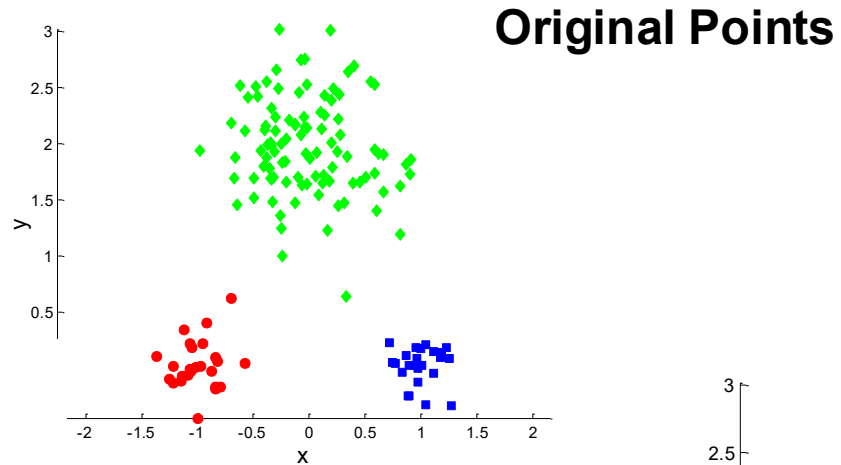
Iterations of K-means Clustering, $k = 3$



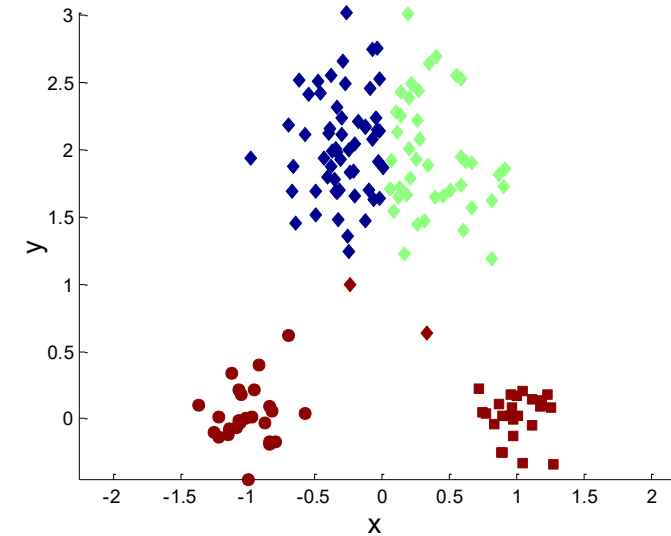
K-means Clustering – Details

- Initial centroids are often chosen randomly.
 - Clusters produced can vary from one run to another
- The centroid is the mean of the points in the cluster.
- K-means will always converge
- Most of the convergence happens in the first few iterations.
 - Often the stopping condition is changed to ‘Until relatively few points change clusters’

K-means does not always converge to optimal solution

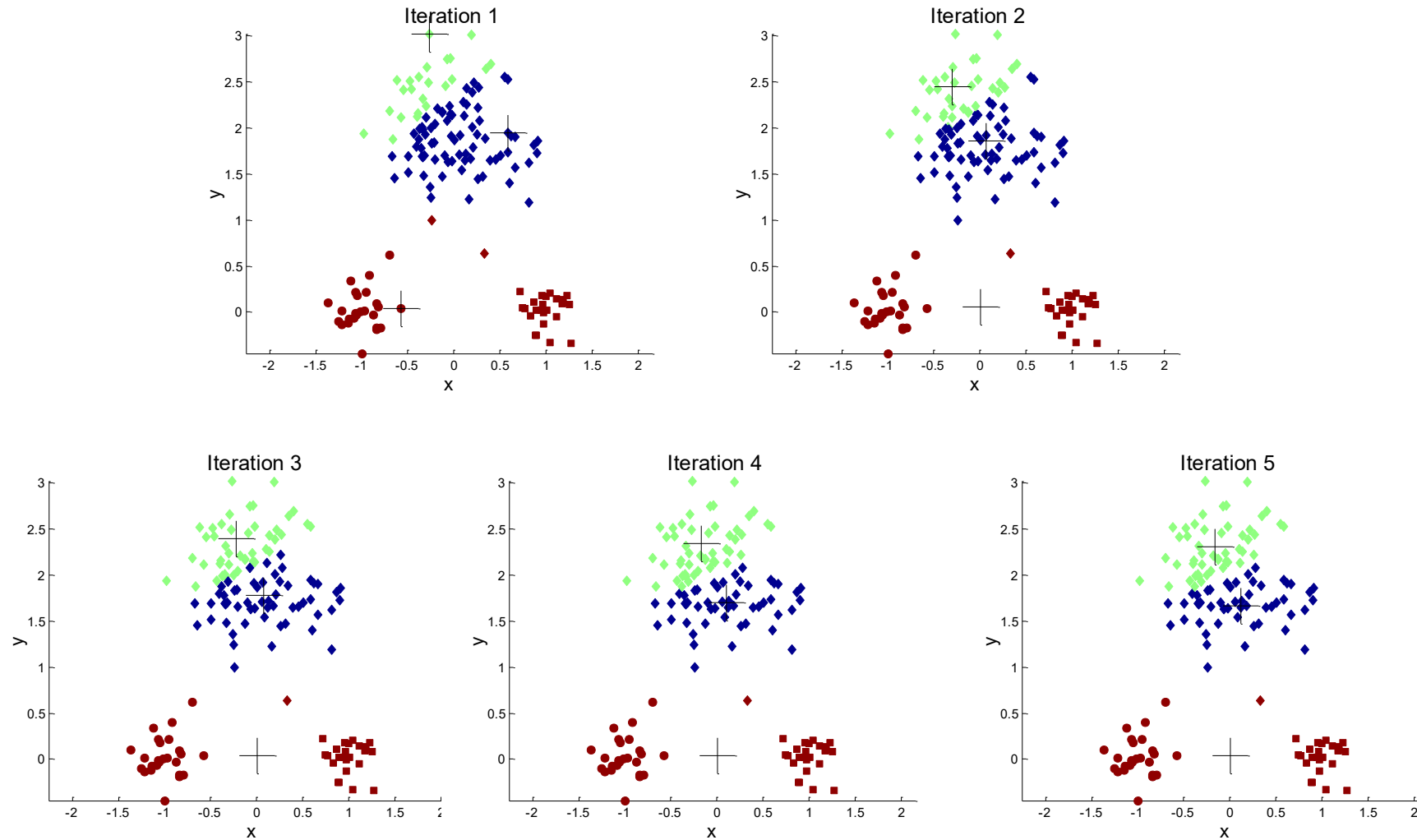


Optimal Clustering



Sub-optimal Clustering

Importance of Choosing Initial Centroids ...



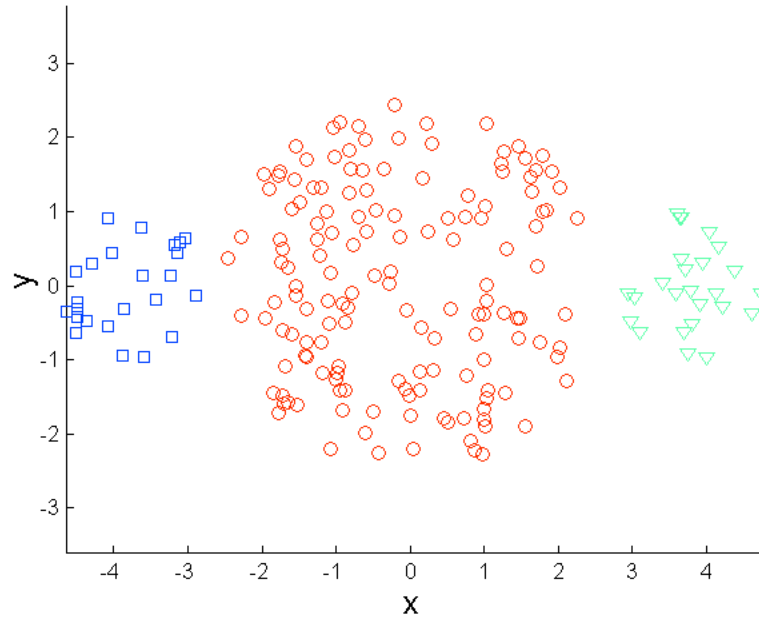
Centroid initialization methods

- Random initialization
- Set it yourself if known
- Select points far away from each other using K++ algorithm - assign the first centroid randomly, then select the rest of the centroids based on the maximum squared distance.
- K++ algorithm is how initial centroids are selected in sklearn

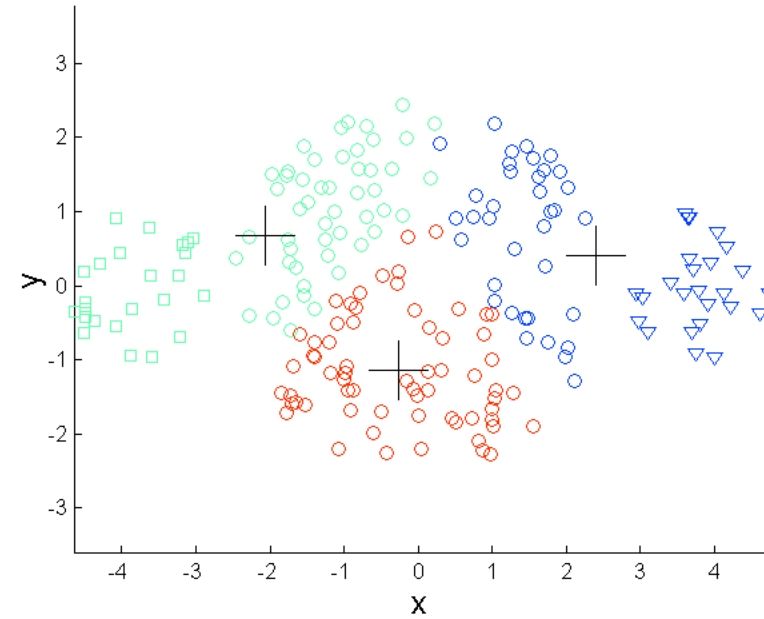
Limitations of K-means

- K-means has problems when clusters are of differing
 - Sizes
 - Densities
 - Non-globular shapes
- K-means has problems when the data contains outliers.

Limitations of K-means: Differing Sizes

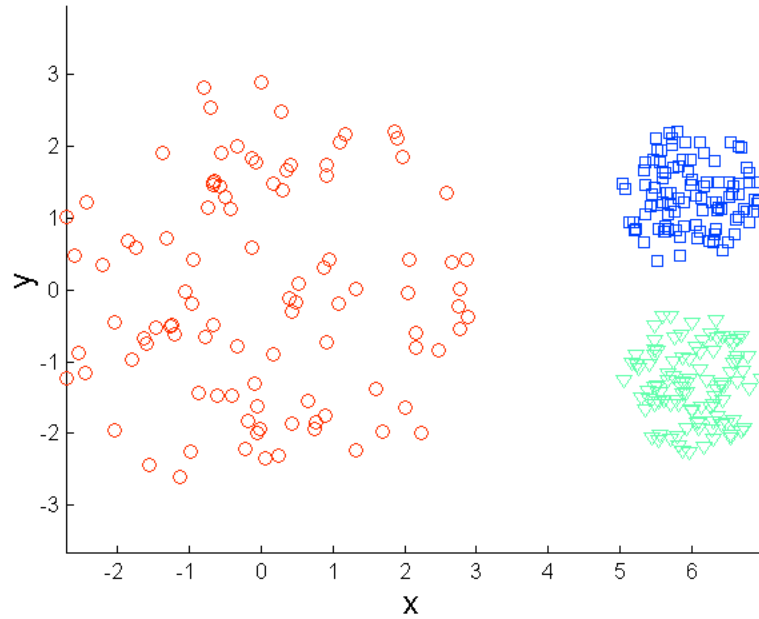


Original Points

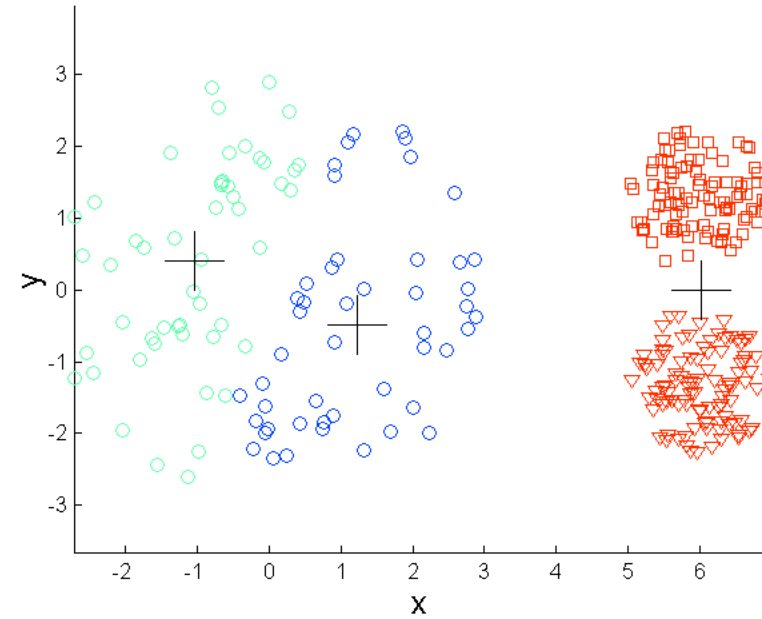


K-means (3 Clusters)

Limitations of K-means: Differing Density

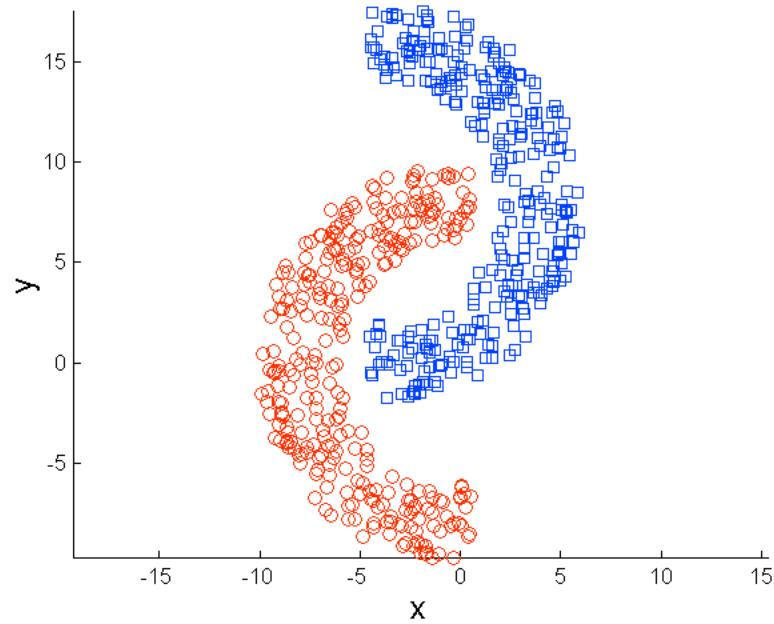


Original Points

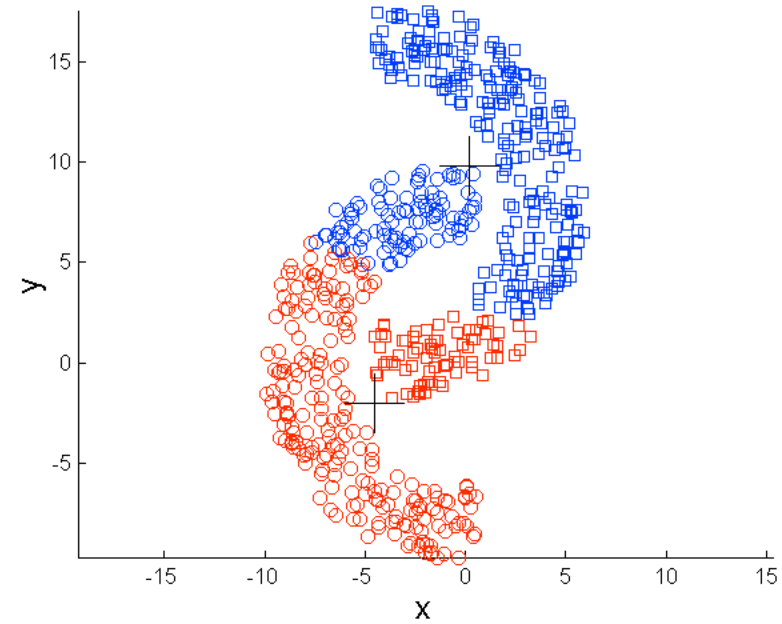


K-means (3 Clusters)

Limitations of K-means: Non-globular Shapes



Original Points



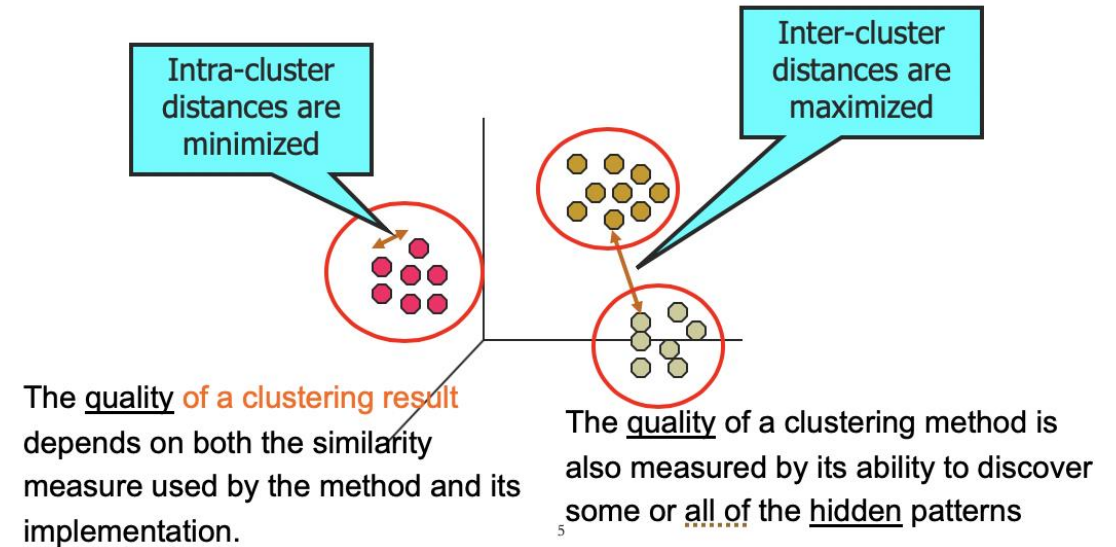
K-means (2 Clusters)

Cluster Validity

- For supervised classification we have a variety of measures to evaluate how good our model is
 - RMSE, correct classification rate, precision, recall
- For cluster analysis, how do we evaluate the “goodness” of the resulting clusters?

Measures of Cluster Validity

- Numerical measures that are applied to judge various aspects of cluster validity, are classified into the following two types.
 - External Index: Used to measure the extent to which cluster labels match externally supplied class labels.
 - Internal Index: Used to measure the goodness of a clustering structure *without* respect to external information.



Internal Measures: inertia

- Inertia - the mean squared distance between each instance and its closest centroid. The lower the inertia the better the cluster.
- BUT issue with inertia – as k increases, inertia decreases.

$$\sum_{i=1}^N (x_i - C_k)^2$$

Silhouette score

The silhouette score is a way to measure how well each data point fits into its cluster. It compares:

- how **close** a point is to its **own cluster** (cohesion), and
- how **far** it is from **other clusters** (separation).

$$s = \frac{b - a}{\max(a, b)}$$

a = average distance to the points in the same cluster (how tightly it fits in its cluster)

b = the mean distance to the instances of the next closest cluster, defined as the one that minimizes b, excluding the instance's own cluster (how far it is from the nearest *other* cluster).

Value can vary between -1 and 1

Typically ranges between 0 and 1.

The closer to 1 the better.

Use Silhouette score to select k

- Don't use inertia to select the best number for k. As k increases, inertia decreases.
- Use silhouette score.

Pre-processing and Post-processing

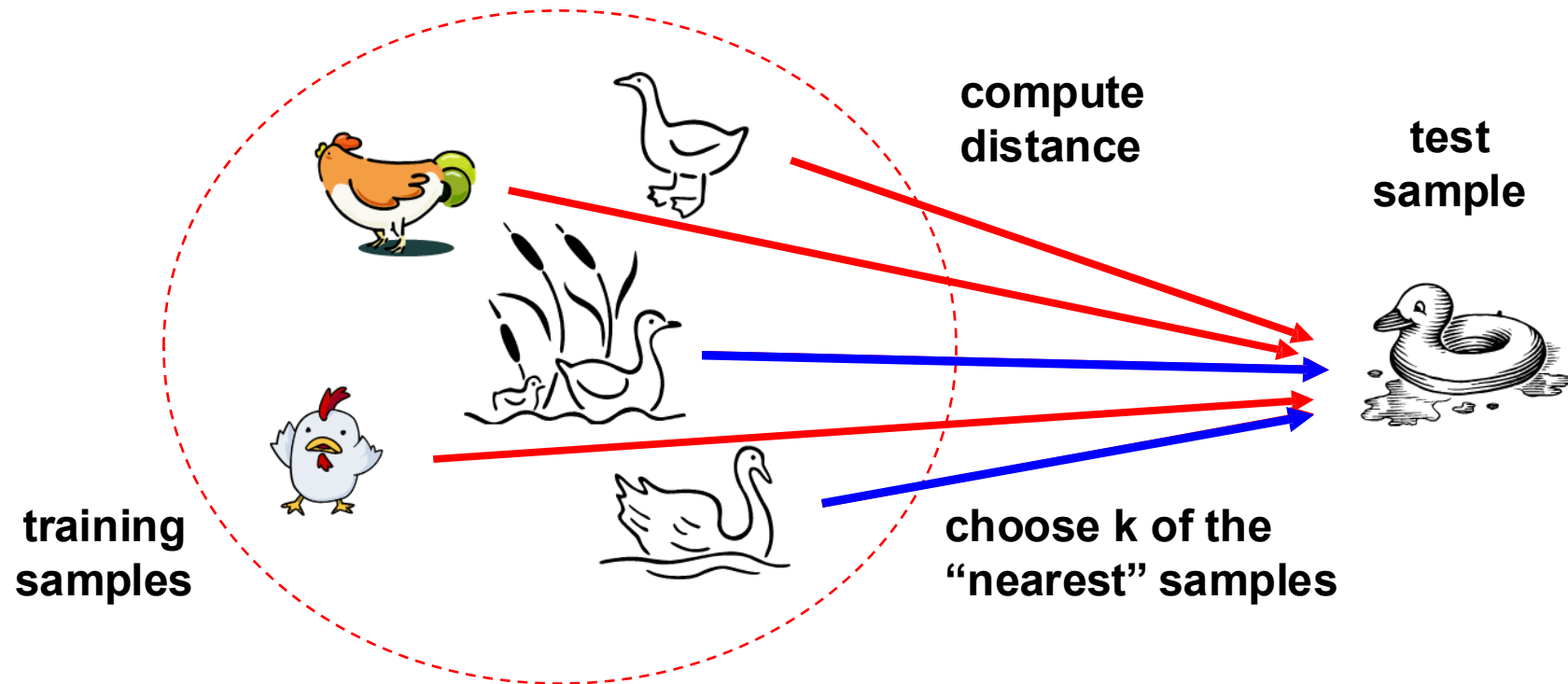
- Pre-processing
 - Scale the data
 - Eliminate outliers
- Post-processing
 - Eliminate empty clusters and small clusters that may represent outliers
 - Split 'loose' clusters, i.e., clusters with relatively high distances between points
 - Merge clusters that are 'close' and that have relatively low distances between points
 - These steps can be used multiple times during the clustering process
- ***Pre-processing must be done by you, post-processing is done by sklearn***

KNN

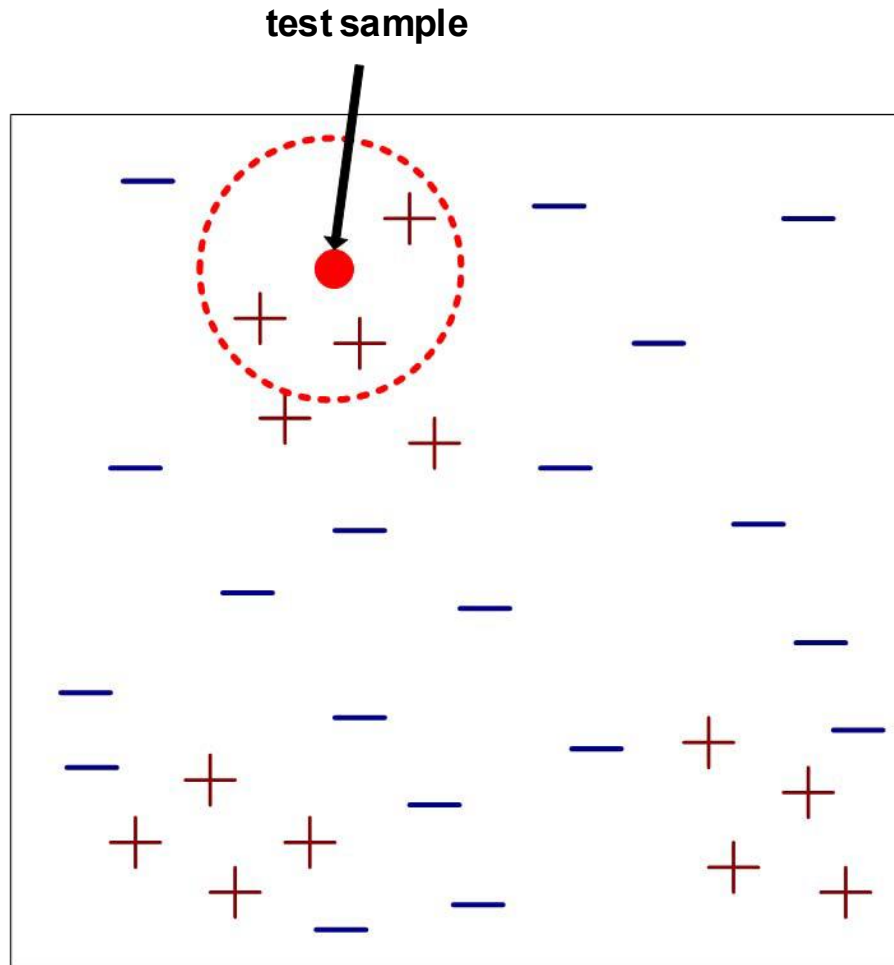
Nearest neighbor classifiers

Basic idea:

If it walks like a duck, quacks like a duck, then
it's probably a duck



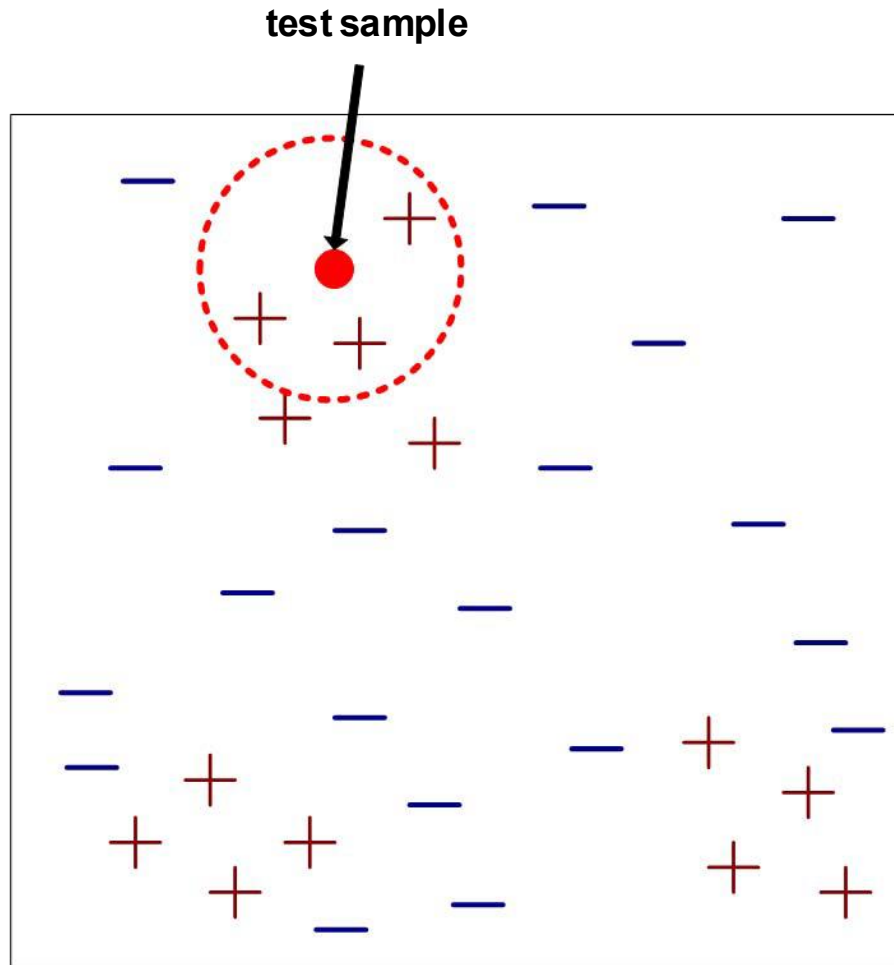
Nearest neighbor classifiers



Requires three inputs:

1. The set of stored samples (training data)
2. Distance metric to compute distance between samples
3. The value of k , the number of nearest neighbors to retrieve

Nearest neighbor classifiers



To classify test sample:

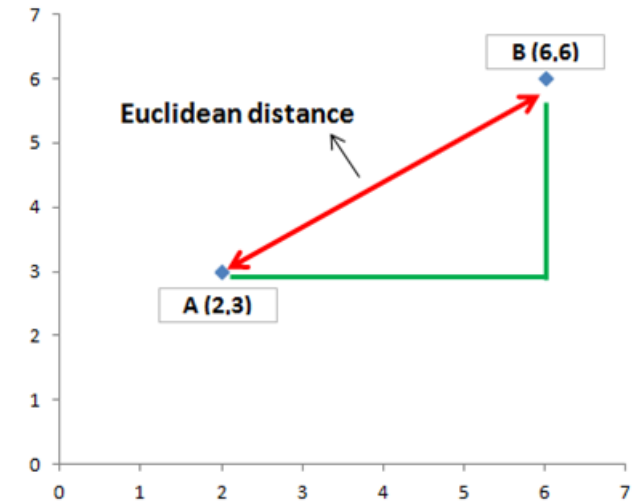
1. Compute distances to samples in training set
2. Identify k nearest neighbors
3. Use class labels of nearest neighbors to determine class label of test sample (e.g. by taking majority vote)

Distance for nearest neighbor classifiers

Option for computing distance between two samples:
Euclidean distance

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_i (x_i - y_i)^2}$$

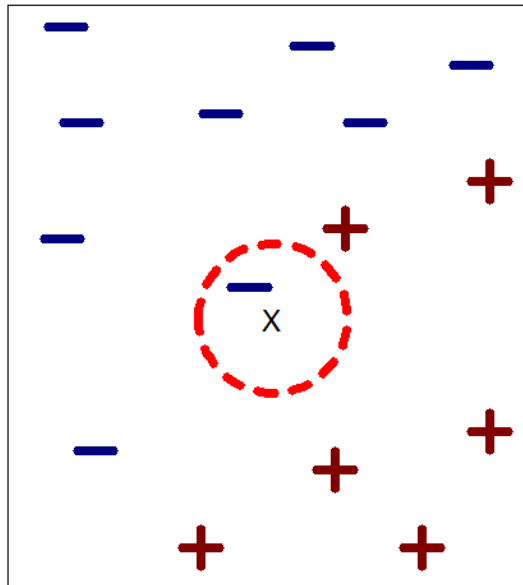
- Scaling issues
 - Attributes may have to be scaled to prevent distance measures from being dominated by one of the attributes



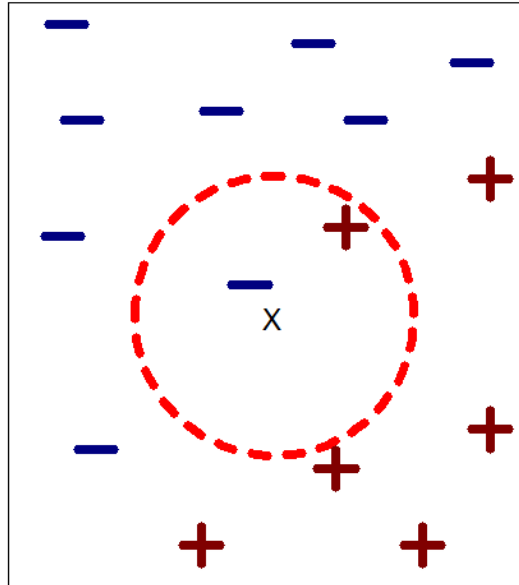
$$\text{Euclidean distance } (a, b) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}$$

Definition of nearest neighbors

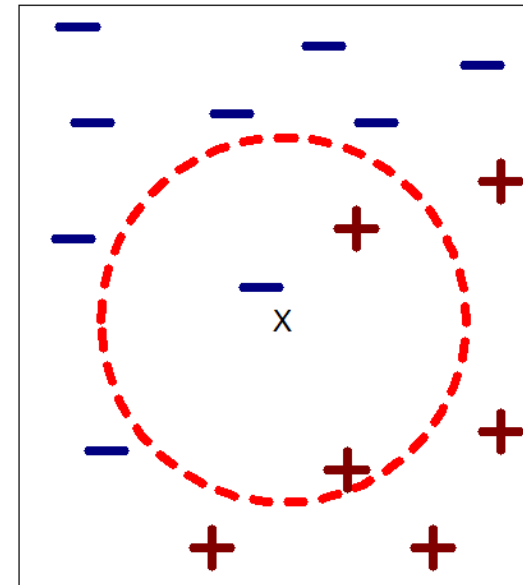
k -nearest neighbors of test sample x are training samples that have the k smallest distances to x



1-nearest neighbor



2-nearest neighbor



3-nearest neighbor

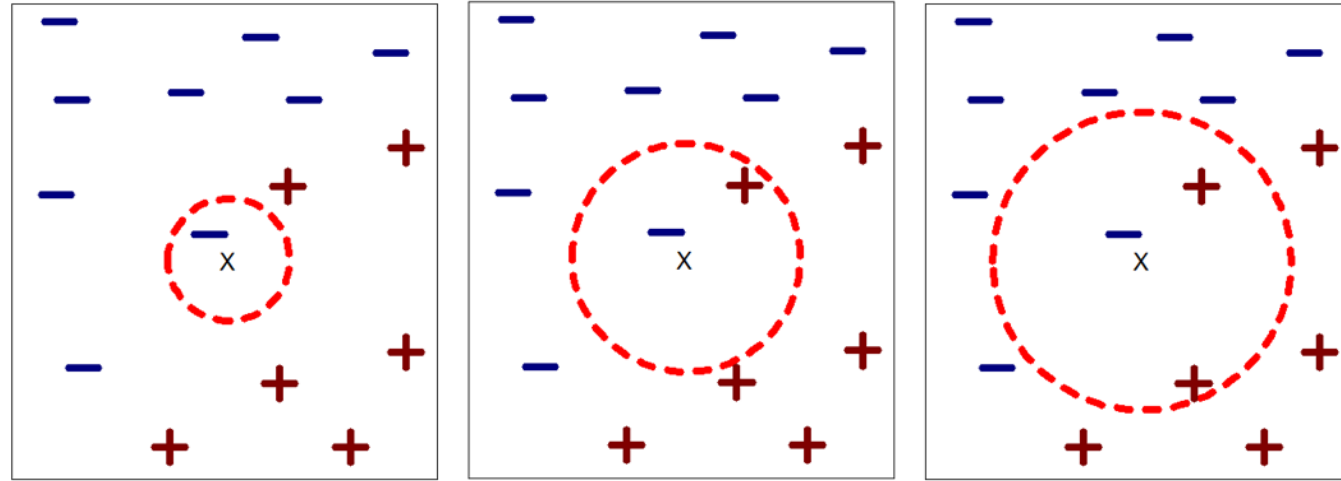
Predicting class from nearest neighbors

Options for predicting test class from nearest neighbor list

- Take majority vote of class labels among the k -nearest neighbors
- Weight the votes according to distance
example: weight factor $w = 1 / d^2$

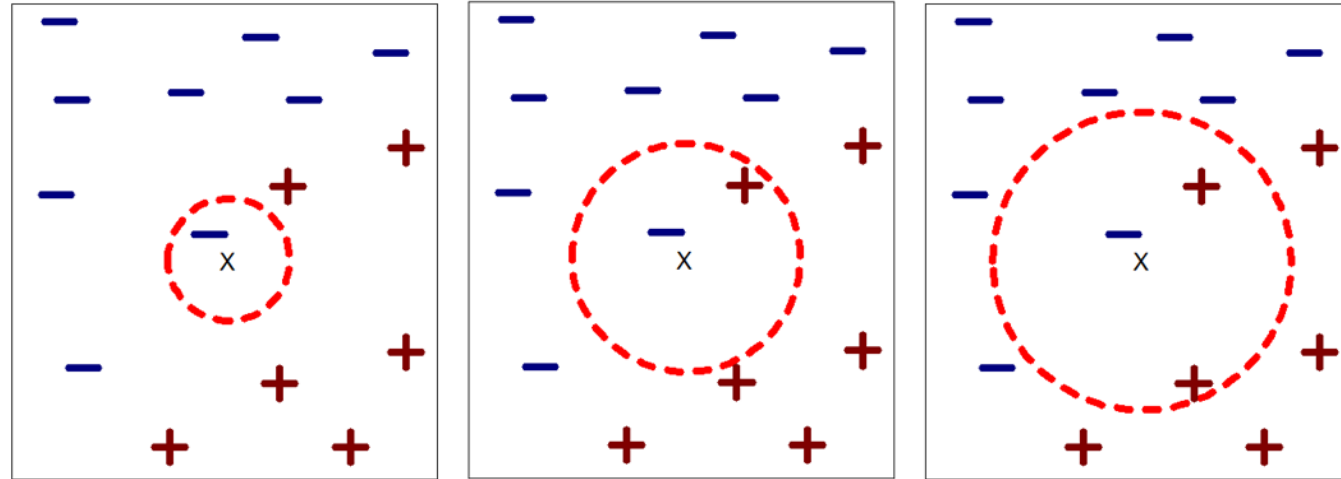
this is an easy way to break ties

Predicting class from nearest neighbors



nearest neighbors	1	2	3
majority vote			
distance-weighted vote			

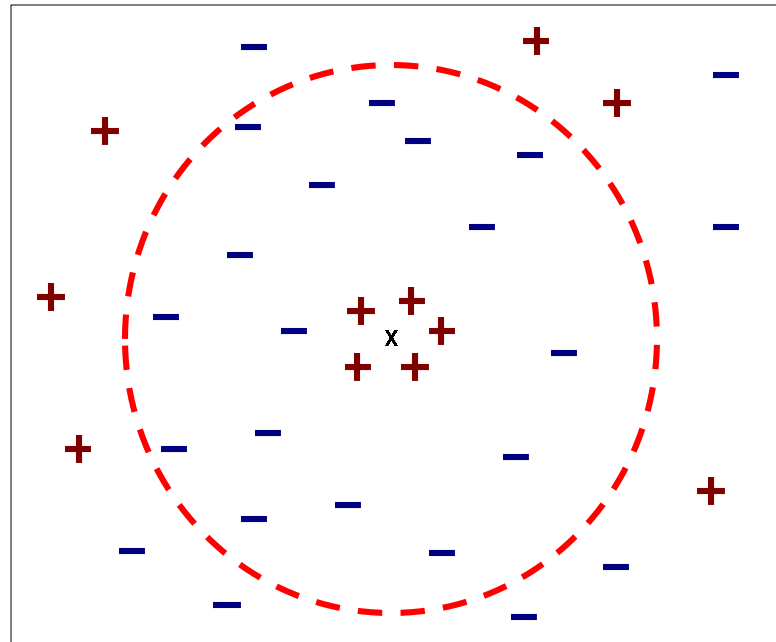
Predicting class from nearest neighbors



nearest neighbors	1	2	3
majority vote	-	+	+
distance-weighted vote	-	-	- or +

Predicting class from nearest neighbors

- Choosing the value of k :
 - If k is too small, sensitive to noise points
 - If k is too large, neighborhood may include points from other classes



Nearest neighbor classification

- k -Nearest neighbor classifier is a **lazy** learner
 - Does not build model explicitly.
 - Classifying unknown samples is relatively expensive.
 - This is in contrast to other machine learning models where training of the model takes time, but predictions happen quickly.
 - Here there is no training, but making the prediction involves computing Euclidean distance which can take time depending on the number of features in dataset.

Nearest neighbor classification

- k -Nearest neighbor can be used for regression problems too
 - the predicted value for the new data point is the average of the target values of the K nearest neighbors

PCA (Principal Component Analysis)

PCA

- Principal component analysis, or PCA, reduces the number of dimensions in large datasets to principal components that retain most of the original information.
- It does this by transforming potentially correlated variables into a smaller set of variables, called principal components.

PCA

PCA finds new axes (called *principal components*) that:

- Capture the most variation in your data.
- Are uncorrelated (don't overlap in what they measure).
- Are sorted by importance (how much information they keep).

Think of it as turning your cloud of data points to a new angle where it looks the flattest — then keeping just the top few dimensions that show the most “spread.”

